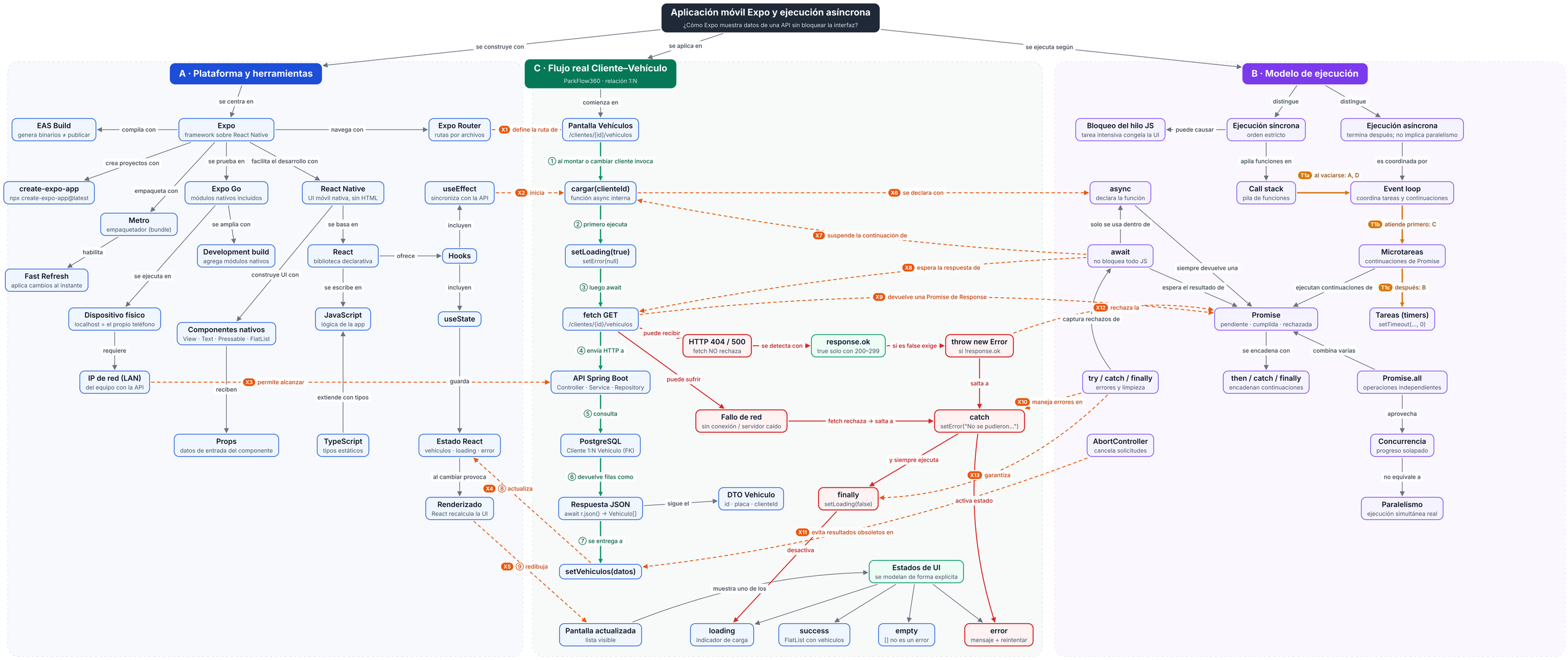
Mapa conceptual · Expo, ejecución síncrona y asíncrona (async/await)

Programación Aplicada · Caso práctico ParkFlow360: Cliente–Vehículo (1:N) · 58 conceptos · 13 enlaces cruzados · 2 recorridos temporales · 1 escenario de fallo

Pregunta focal: ¿Cómo Expo permite mostrar datos de una API sin bloquear la interfaz?



Leyenda

- A · Plataforma y herramientas
- B · Modelo de ejecución
- C · Flujo real Cliente–Vehículo
- Escenario de error / control de fallos
- Relación jerárquica (general → particular)
- Recorrido T2: consulta completa ①→⑨
- Recorrido T1: síncrono → asíncrono
- Camino de error
- Enlace cruzado entre ramas (X1–X13)

Cada flecha se lee como oración: *concepto —palabra de enlace→ concepto*.

T1 Ejemplo 1 · Orden de ejecución

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
// Salida típica: A, D, C, B
```

1. **T1a** A y D son síncronos: se ejecutan en el call stack hasta vaciarlo.
2. **T1b** El event loop atiende primero las microtareas: la continuación de la Promise imprime C.
3. **T1c** Después corre la tarea del temporizador: B. `setTimeout(…, 0)` significa «no antes de 0 ms», no «inmediato».

T2 Ejemplo 2 · Consulta Cliente–Vehículo

```
async function obtenerVehiculos(clienteId) {
  const r = await fetch(`${API_URL}/clientes/${clienteId}/vehiculos`);
  if (!r.ok) throw new Error(`HTTP ${r.status}`);
  return await r.json();
}
async function cargar(clienteId) {
  setLoading(true); setError(null);
  try { setVehiculos(await obtenerVehiculos(clienteId)); }
  catch (e) { setError("No se pudieron cargar vehículos"); }
  finally { setLoading(false); }
}
```

Pantalla → useEffect → `cargar()` async → fetch → API Spring Boot → PostgreSQL → JSON → `setVehiculos` → estado React → renderizado → pantalla. (URL ilustrativa: verificar endpoint en Swagger.)

Escenario de fallo y Ejemplo 3

- **Red caída:** fetch *rechaza* → `catch` → `setError` → estado **error**; `finally` apaga loading.
- **HTTP 404/500:** fetch *no* rechaza → `!response.ok` → `throw` → `catch`.
- **Lista vacía:** respuesta 200 con `[]` → estado **empty**, no error.
- **Cliente cambia antes de terminar:** AbortController / bandera `activo` evita sobrescribir con datos obsoletos.

```
// Ejemplo 3: secuencial vs concurrente
const c = await obtenerCliente(id);          // depende
const v = await obtenerVehiculos(c.id);      // de la anterior
const [cli, veh] = await Promise.all([       // independientes:
  obtenerCliente(id), obtenerVehiculos(id)]); // en concurrencia
```

Enlaces cruzados (léanse en voz alta)

- X1 **Expo Router** —define la ruta de→ **Pantalla Vehículos**
- X2 **useEffect** —inicia→ **cargar(clienteId)**
- X3 **IP de red (LAN)** —permite alcanzar→ **API Spring Boot**
- X4 **setVehiculos(datos)** —actualiza→ **Estado React**
- X5 **Renderizado** —redibuja→ **Pantalla actualizada**
- X6 **cargar(clienteId)** —se declara con→ **async**
- X7 **await** —suspende la continuación de→ **cargar(clienteId)**
- X8 **await** —espera la respuesta de→ **fetch GET**
- X9 **fetch GET** —devuelve una Promise de Response→ **Promise**
- X10 **try / catch / finally** —maneja errores en→ **catch**
- X11 **AbortController** —evita resultados obsoletos en→ **setVehiculos(datos)**
- X12 **throw new Error** —rechaza la→ **Promise**
- X13 **try / catch / finally** —garantiza→ **finally**

Errores a evitar: Expo no es un lenguaje · React ≠ React Native · async no paraleliza · await no bloquea toda la app · useEffect no recibe directamente una función async · 1:N no garantiza que existan vehículos · en un teléfono físico, localhost no apunta a la computadora.